

GraphCompose Markdown

User Manual — written in Markdown, rendered by the library itself

Artem Demchyshyn · 2026-07-01

Contents

Contents	1
What is this?	1
Installation	2
Quick start	2
Ways in, ways out	2
What renders	3
Inline formatting	3
Smart punctuation	3
Lists and tasks	3
Tables	3
Code with syntax highlighting	4
Quotes, alerts and callouts	4
Emoji	4
Theming	4
Navigation and page chrome	5
Extending	5
Command line	5
Runnable examples	6
How this PDF was made	6

What is this?

graphcompose-markdown turns Markdown into themed, paginated PDF. It parses with [Flexmark](#), maps the parse tree onto an independent semantic model, and renders that model through the [GraphCompose](#) engine, which owns measurement, layout, pagination and output.

Three concerns stay separate, and that separation is the whole point:

- Content — your Markdown text.
- Appearance — a theme: colors, fonts, spacing, per-element styles, renderers.
- Layout — the engine: measurement, pagination, PDF.

The same Markdown can be reskinned into a completely different document without touching its

text. The document you are reading right now is the proof: it is a plain `.md` file rendered by the library, using the book-style table of contents above (with live page numbers), the “Page N of M” footer below, vector emoji, and smart punctuation — every dash, ellipsis and curly quote in this manual was typed as `--`, `...` and straight quotes.

 Note

Everything shown in this manual is produced by the code paths it documents. If a feature renders here, it works.

Installation

The library is a single Maven dependency; the GraphCompose engine comes in transitively:

```
<dependency>
  <groupId>io.github.demchaav</groupId>
  <artifactId>graph-compose-markdown</artifactId>
  <version>0.3.0</version>
</dependency>
```

Two optional companion artifacts unlock extras; add them only if you want them:

Artifact	What it adds
<code>graph-compose-fonts</code>	JetBrains Mono for code (<code>BundledFonts.jetbrainsMonoCode(theme)</code>)
<code>graph-compose-emoji</code>	Noto vector emoji: <code>:rocket:</code> just works 🚀

Requires Java 17+.

Quick start

```
import io.github.demchaav.markdown.composer.MarkdownComposer;
import io.github.demchaav.markdown.theme.DefaultMarkdownTheme;
import java.nio.file.Path;

MarkdownComposer composer = MarkdownComposer.create(DefaultMarkdownTheme.light());

composer.render("# Hello\n\nWorld").writePdf(Path.of("hello.pdf"));
composer.renderFile(Path.of("doc.md")).writePdf(Path.of("doc.pdf"));
```

A composer is immutable and thread-safe — build it once, reuse it everywhere.

Ways in, ways out

Take this in	Call
A Markdown string	<code>render(String)</code>
A Markdown file (UTF-8, relative images resolve next to it)	<code>renderFile(Path)</code>
A file with your own image resolver	<code>renderFile(Path, ImageResolver)</code>

A parsed Flexmark <code>Document</code>	<code>render(Document)</code>
A hand-built semantic model	<code>render(MarkdownDocument)</code>

Get this out	Call
A PDF file	<code>writePdf(Path)</code>
A PDF stream	<code>writePdf(OutputStream)</code>
PDF bytes	<code>toPdfBytes()</code>
Page images (thumbnails, previews)	<code>toImages(dpi)</code> / <code>toImage(pageIndex, dpi)</code>

What renders

Everything in this section is live — the source of this manual uses each feature.

Inline formatting

Paragraphs carry bold, italic, strikethrough and `inline code` on its rounded chip. Links come in three kinds: [external](#), bare autolinks like <https://github.com>, and internal jumps — [back to the contents](#) is a native PDF go-to action, not decoration. Footnotes are clickable in both directions.^[1]

Smart punctuation

This manual is rendered with `smartPunctuation(true)`: dashes – and — em-dashes, ellipses... and “curly quotes” are typed as plain ASCII in the source. It is off by default (GitHub does not smart-quote); code always stays verbatim: `a --flag "x"`.

Lists and tasks

1. Ordered lists keep their numbering
2. and nest:
 - unordered bullets cycle shape by depth
 - task lists render real checkboxes:
 - ☒ parse Markdown
 - ☒ map to the semantic model
 - ☐ profit

Tables

Feature	Alignment	Status
GFM tables	center	☒
Per-column alignment	center	☒
Inline formatting in cells	works	<code>yes</code>

Code with syntax highlighting

```
// The built-in RegexSyntaxHighlighter covers ~15 languages, no extra dependency.
public static void main(String[] args) {
    var composer = MarkdownComposer.create(GitHubTheme.dark());
    byte[] pdf = composer.render("# Reskinned").toPdfBytes();
    // 42 ways to say hello
}
```

Quotes, alerts and callouts

A blockquote is a left-accented aside. Design tokens drive its cosmetics.

Tip

The five GitHub alerts (`NOTE` , `TIP` , `IMPORTANT` , `WARNING` , `CAUTION`) render as color-coded panels with vector icons.

Warning

Their accent colors are theme tokens (`AlertColors`) — a dark theme can retune them.

`:::` custom blocks are an extension seam: register your own renderer per block type, or let unbound types fall back to this callout style.

Emoji

With `graph-compose-emoji` on the classpath, shortcodes render as crisp vector glyphs at any size: 🚀✅❤️🌟🔥 — no image files to supply. Geometric emoji typed literally render as native vector shapes even without it: status ● blocker, ● important, ● nice to have — and inside code too:

Priority: ● blocker ● important ● nice to have

An unknown shortcode stays readable text: `:definitelynotanemoji:`.

Theming

A theme has three layers; override exactly as much as you need:

1. Design tokens (`MarkdownTokens`) — colors, fonts, sizes, spacing, page geometry.
2. Component styles (`MarkdownStyles`) — per-element styles derived from tokens.
3. Node renderers (`NodeRenderer`) — the behaviour turning each node into engine calls.

```
MarkdownTheme base = DefaultMarkdownTheme.light();
MarkdownTheme custom = MarkdownTheme.builder(base)
    .tokens(base.tokens().withColors(
        base.tokens().colors().withCodeBackground(DocumentColor.rgb(246
, 248, 250))))
    .renderer(CodeBlockNode.class, new MyLabeledCodeBlockRenderer())
    .build();
```

Ready-made packs ship in `io.github.demchaav.markdown.theme.packs : GitHubTheme` (light/dark), `AcademicTheme`, `MinimalTheme`, `BusinessReportTheme` — plus `DefaultMarkdownTheme.light()` / `.dark()`.

Navigation and page chrome

Long documents stay navigable; this manual demonstrates all of it:

- PDF outline — every heading becomes a bookmark; the panel opens automatically (opt out: `builder().openOutline(false)`).
- Anchors — every heading gets a GitHub-style slug, so `[text](#what-renders)` jumps [there](#).
- `[TOC]` — the marker at the top of this manual expands into the contents. Two forms: the default clickable link list, and the book form used here — dot leaders and live page numbers, resolved by the engine after layout:

```
MarkdownTheme book = MarkdownTheme.builder(base)
    .renderer(TocNode.class, new BookTocRenderer("Contents"))
    .tokens(base.tokens().withFooter(FooterTokens.pageNumbers()))
    .build();
```

- Footer — the “Page N of M” at the bottom of every page here is `FooterTokens : left / center / right` templates with `{page}`, `{pages}` and `{date}` placeholders, disabled by default.

Extending

- Custom `:::` blocks — `builder().customBlock("chart", new ChartRenderer())` routes every `:::chart` block to your renderer.
- Swap any renderer — `builder().renderer(TableNode.class, myTableRenderer)` replaces one node type and reuses everything else (that is exactly how the book TOC works).
- Emoji override — an `EmojiResolver` supplying PNG bytes wins over the vector set.
- Syntax highlighting — plug a grammar-based `SyntaxHighlighter` via `builder().highlighter(...)`; colors stay theme tokens.
- Strict mode — `builder().strictMode(true)` rejects unsupported content (raw HTML) instead of surfacing it as literal text.

Command line

The standalone `cli/` module wraps all of this for the shell:

```
gcmd README.md -t github-dark      # theme by name
gcmd docs/guide.md -o out/guide.pdf # explicit output
cat notes.md | gcmd - -o notes.pdf  # stdin
```

Runnable examples

Each feature ships a runnable example under `examples/` — see `examples/README.md` :
QuickStart, RenderMarkdownFile, ThemeGallery, CustomBlock, AlertsAndOutline,
InPdfNavigation, Toc, BookToc, Footer, PngExport, VectorEmoji, Emoji, FrontMatter.

How this PDF was made

This manual is `assets/readme/manual.md` , rendered by `ManualTest` with exactly this composer
— the self-referential proof that the pipeline works:

```
MarkdownTheme base = DefaultMarkdownTheme.light();
MarkdownTheme manualTheme = MarkdownTheme.builder(base)
    .renderer(TocNode.class, new BookTocRenderer())
    // page-numbered contents
    .tokens(base.tokens().withFooter(FooterTokens.pageNumbers()))
    .imageResolver(new DefaultImageResolver(Path.of("assets", "readme")))
    .build();

MarkdownComposer.builder()
    .theme(manualTheme)
    .smartPunctuation(true)
    // the dashes you saw
    .build()
    .renderFile(Path.of("assets", "readme", "manual.md"))
    .writePdf(Path.of("assets", "readme", "manual.pdf"));
```

Both libraries are MIT-licensed. Happy composing. 🎉

Notes

[\[1\]](#) Click the marker above to land here; click the `[1]` at the start of this note to jump back to the citation.